Neural Networks : Supervised Learning Approach

Dr. Satyasai Jagannath Nanda
Dept. of Electronics & Communication Engineering
Malaviya National Institute of Technology Jaipur
Email : sjnanda.ece@mnit.ac.in

Contents

- Introduction and Inspiration
- Single Input neuron
- Multiple input neuron
- A layer of neuron
- Multi layer perceptron
- ADLINE
- Backpropagation
- FLANN
- RBF

Biological Inspiration

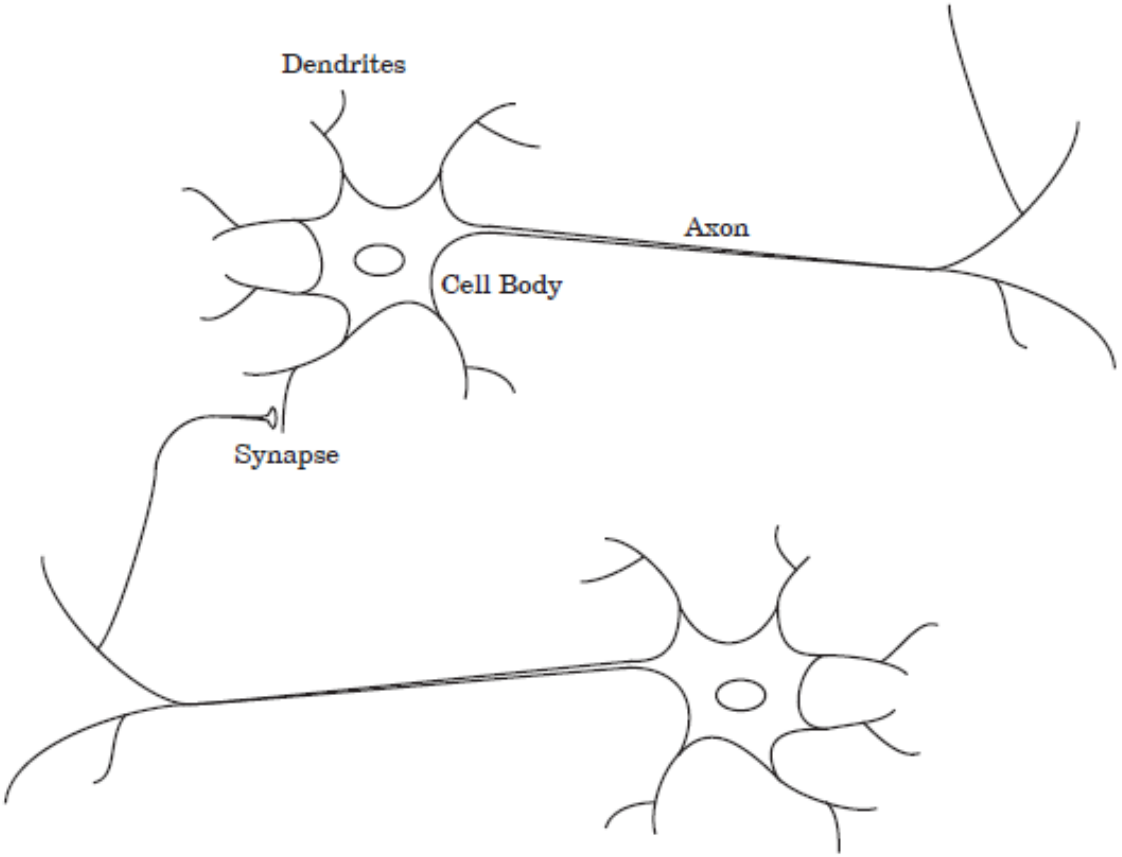
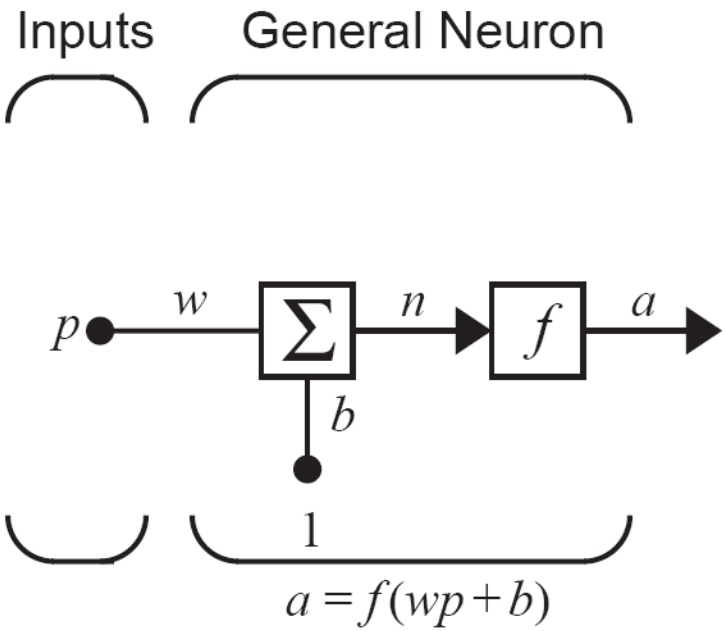


Figure: Schematic drawing of biological neurons

- What is a Neural Network?
  - A method of computing, based on the interaction of multiple connected processing elements

- What can a Neural Net do?
  - Compute a known function
  - Approximate an unknown function
  - Pattern Recognition
  - Signal Processing

Single Input Neuron



Resemblance biological Neuron and Neuron Model

Strength of Synapse	Weight
Transfer function and summation	Cell body
Signal on axon	Output

f = Transfer function or activation function

- One can choose neurons with or without biases. The bias gives the network an extra variable, and so you might expect that networks with biases would be more powerful than those without bias

- The bias b is like a weight, except that it has constant input 1.

- ***w*** and ***b*** are both ***adjustable*** parameters of the neuron.

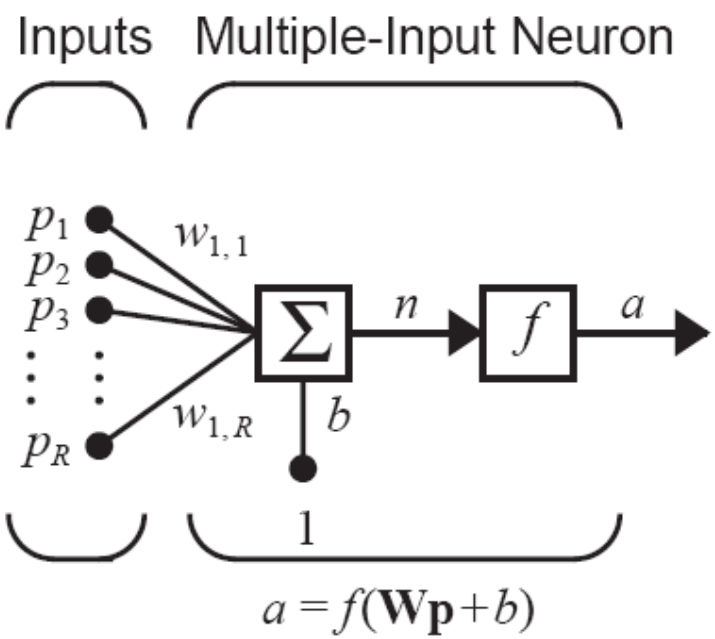
- *The **transfer function** may be **linear** or **non linear***

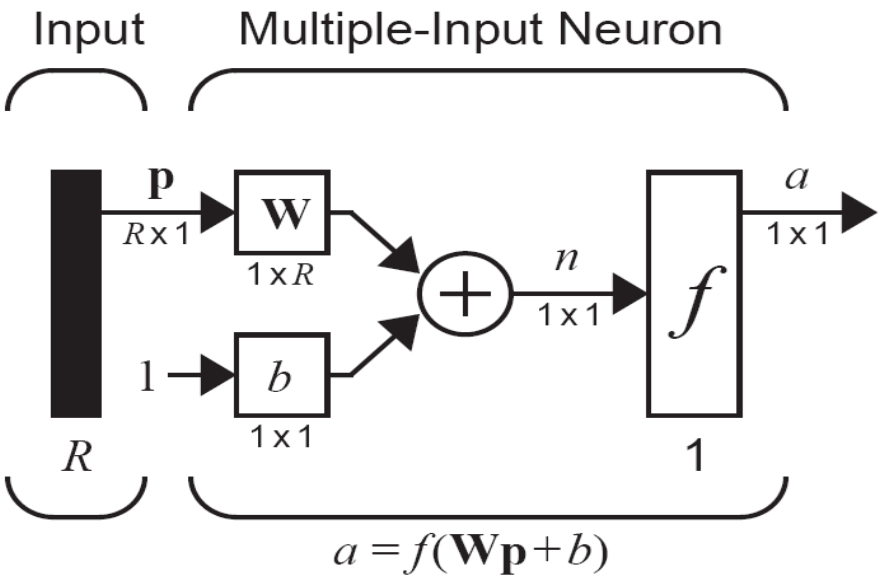
- The transfer function is chosen by the designer and then the parameters will be adjusted by some learning rule so that the neuron input/output relationship meets some specific goal.

Nonlinear functions

Name	Input/Output Relation	Icon	MATLAB Function
Hard Limit	$a = 0 \quad n < 0$ $a = 1 \quad n \geq 0$		hardlim
Symmetrical Hard Limit	$a = -1 \quad n < 0$ $a = +1 \quad n \geq 0$		hardlims
Linear	$a = n$		purelin
Saturating Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n \leq 1$ $a = 1 \quad n > 1$		satlin
Symmetric Saturating Linear	$a = -1 \quad n < -1$ $a = n \quad -1 \leq n \leq 1$ $a = 1 \quad n > 1$		satlins
Log-Sigmoid	$a = \frac{1}{1 + e^{-n}}$		logsig
Hyperbolic Tangent Sigmoid	$a = \frac{e^{n} - e^{-n}}{e^{n} + e^{-n}}$		tansig
Positive Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n$		poslin
Competitive	$a = 1$ neuron with max n $a = 0$ all other neurons		compet

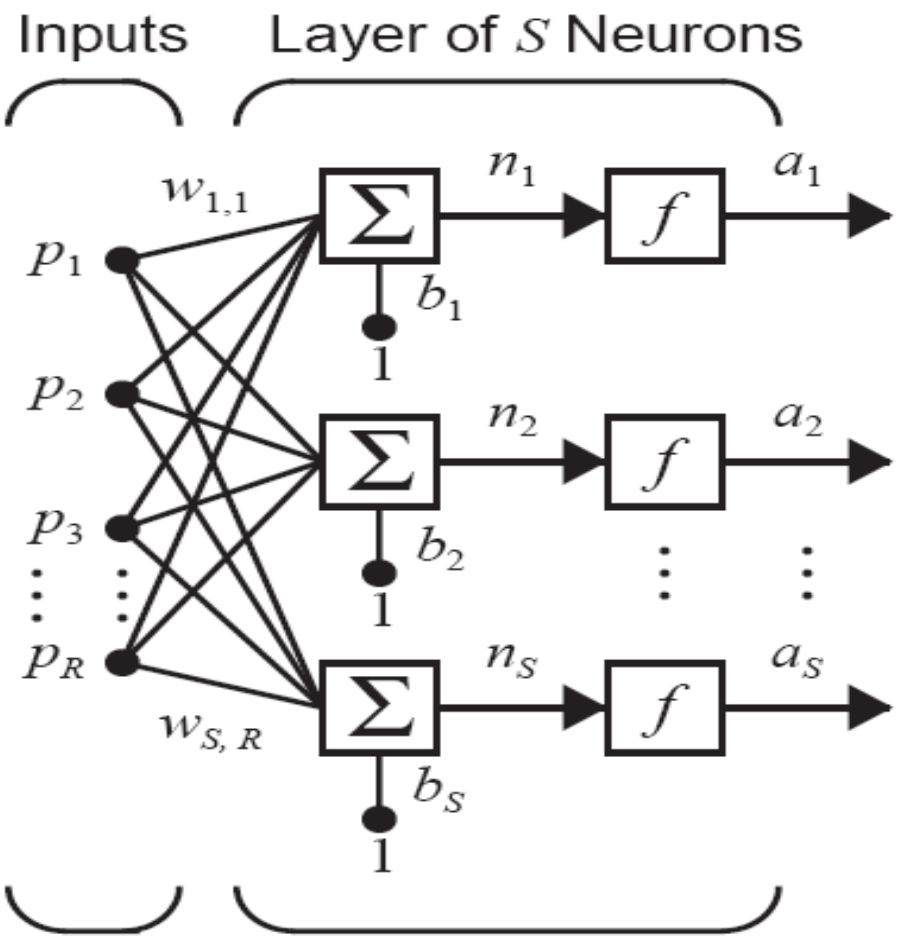
Multiple-Input Neuron





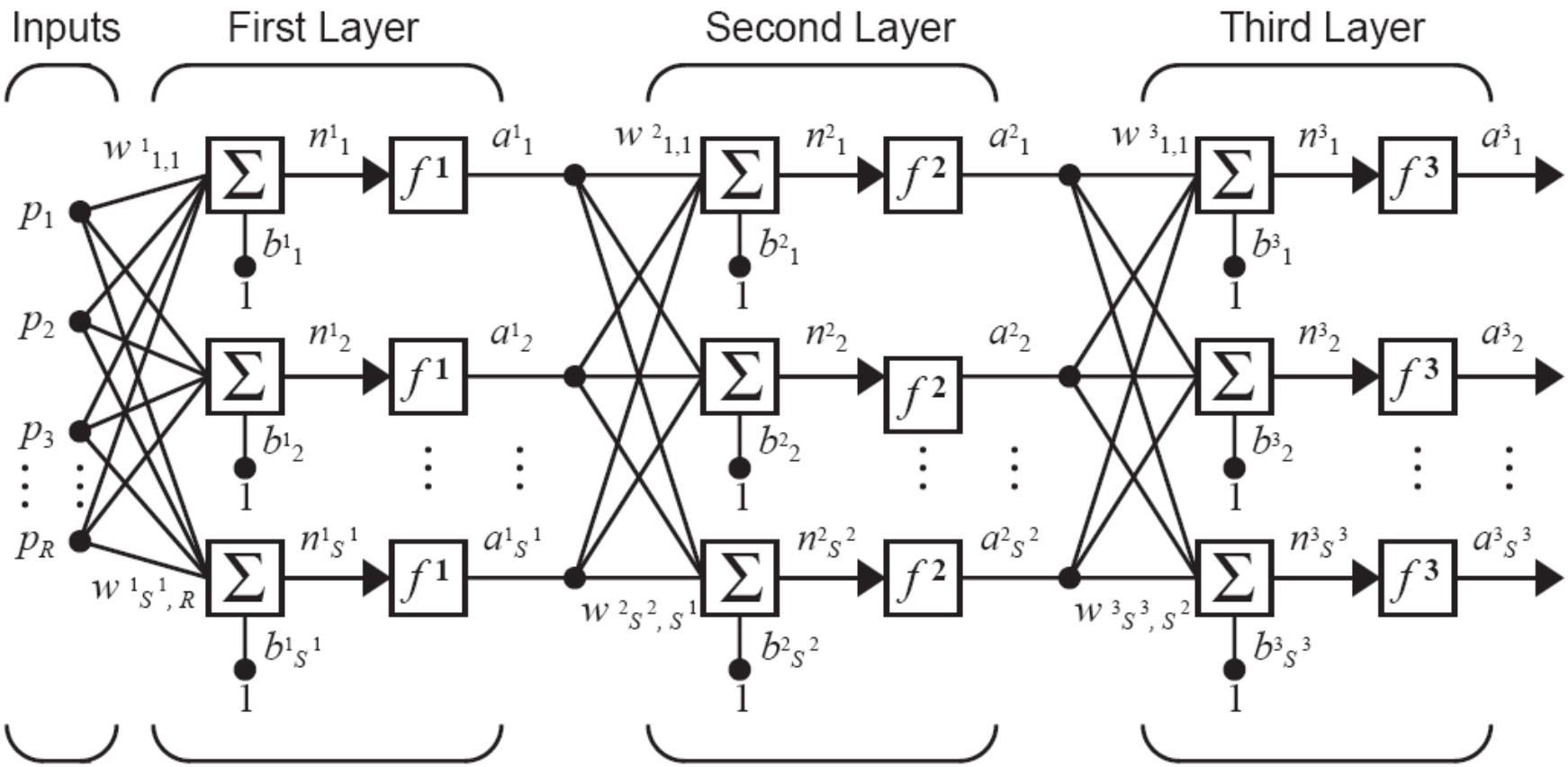
The first index indicates the particular neuron destination for that weight. The second index indicates the source of the signal fed to the neuron

A layer of Neurons



$$\mathbf{a} = \mathbf{f}(\mathbf{W}\mathbf{p} + \mathbf{b})$$

Multiple Layer of Neurons

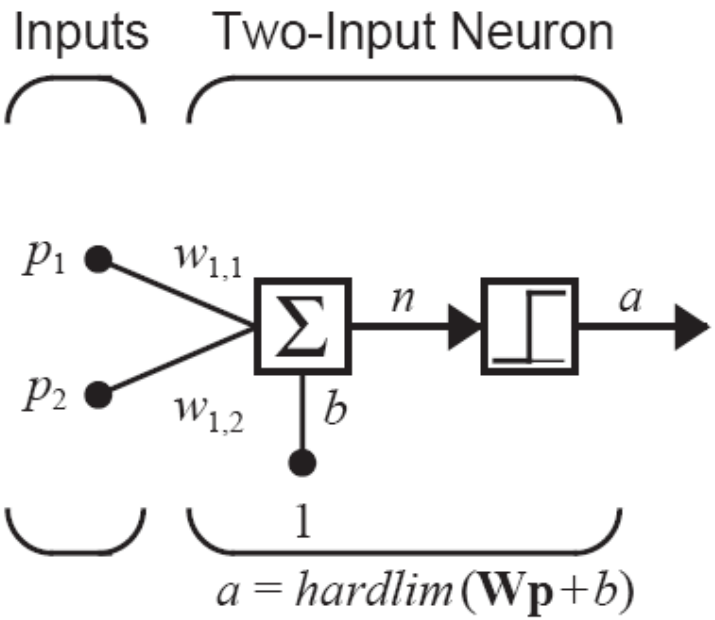


$$\mathbf{a}^1 = \mathbf{f}^1(\mathbf{W}^1\mathbf{p}+\mathbf{b}^1) \qquad \mathbf{a}^2 = \mathbf{f}^2(\mathbf{W}^2\mathbf{a}^1+\mathbf{b}^2) \qquad \mathbf{a}^3 = \mathbf{f}^3(\mathbf{W}^3\mathbf{a}^2+\mathbf{b}^3)$$

$$\mathbf{a}^3 = \mathbf{f}^3(\mathbf{W}^3\mathbf{f}^2(\mathbf{W}^2\mathbf{f}^1(\mathbf{W}^1\mathbf{p}+\mathbf{b}^1)+\mathbf{b}^2)+\mathbf{b}^3)$$

Perceptron Learning Rule

Learning rule means a procedure for modifying the weights and biases of a network.



$$a = hardlim(n) = hardlim(\mathbf{W}\mathbf{p} + b)$$
$$= hardlim({}_1\mathbf{w}^T\mathbf{p} + b) = hardlim(w_{1,1}p_1 + w_{1,2}p_2 + b)$$

Decision Boundary

The *decision boundary* is determined by the input vectors for which the net Input *n* is zero:

$$n = {}_1\mathbf{w}^T\mathbf{p} + b = w_{1,1}p_1 + w_{1,2}p_2 + b = 0 .$$

$$w_{1,1} = 1 , w_{1,2} = 1 , b = -1 .$$

$$n = p_1 + p_2 - 1 = 0 .$$

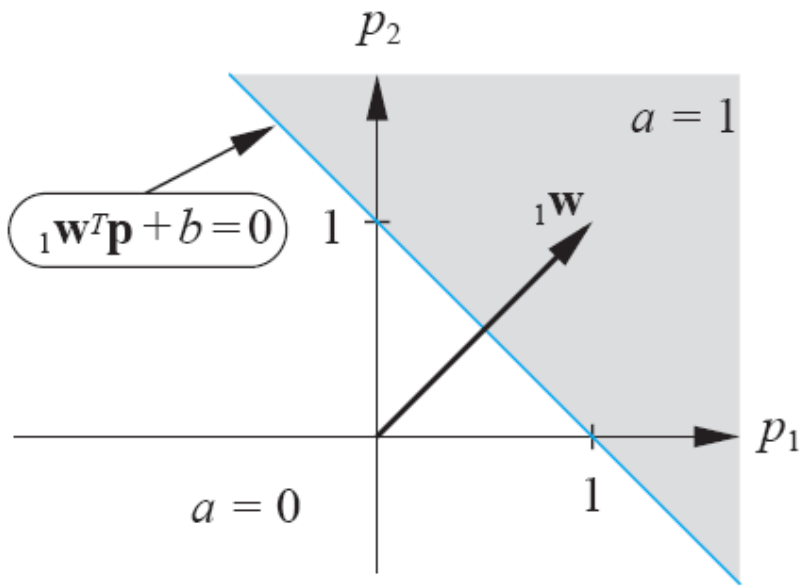
$$p_2 = -\frac{b}{w_{1,2}} = -\frac{-1}{1} = 1 \qquad \text{if } p_1 = 0 .$$

$$p_1 = -\frac{b}{w_{1,1}} = -\frac{-1}{1} = 1 \qquad \text{if } p_2 = 0 .$$

$$\mathbf{p} = \begin{bmatrix} 2 & 0 \end{bmatrix}^T$$

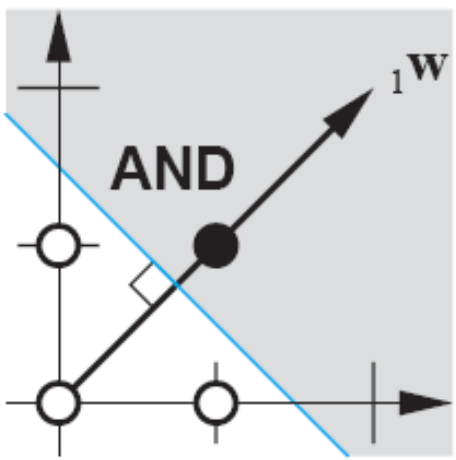
$$a = hardlim({}_1\mathbf{w}^T\mathbf{p} + b)$$

$$= hardlim\left(\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} - 1 \right) = 1 .$$



Example: AND Gate

$$\left\{\mathbf{p}_1 = \begin{bmatrix}0\\0\end{bmatrix}, t_1 = 0\right\} \left\{\mathbf{p}_2 = \begin{bmatrix}0\\1\end{bmatrix}, t_2 = 0\right\} \left\{\mathbf{p}_3 = \begin{bmatrix}1\\0\end{bmatrix}, t_3 = 0\right\} \left\{\mathbf{p}_4 = \begin{bmatrix}1\\1\end{bmatrix}, t_4 = 1\right\}$$



$$_1\mathbf{w} = \begin{bmatrix}2\\2\end{bmatrix},$$

$$\mathbf{p} = \begin{bmatrix}1.5 & 0\end{bmatrix}^T$$

$$_1\mathbf{w}^T\mathbf{p} + b = \begin{bmatrix}2 & 2\end{bmatrix}\begin{bmatrix}1.5\\0\end{bmatrix} + b = 3 + b = 0 \quad \Rightarrow \quad b = -3.$$

Testing

$$a = hardlim({}_1\mathbf{w}^T\mathbf{p}_2 + b) = hardlim\left(\begin{bmatrix}2 & 2\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} - 3\right)$$

$$a = hardlim(-1) = 0,$$

All other points can be tested in this way

A single-neuron Perceptron can classify input vectors into two categories, since its output can be either 0 or 1.

If the number of neurons is S then there will be 2^S possible categories..

Perceptron Learning Rule

Supervised Learning

Reinforcement learning

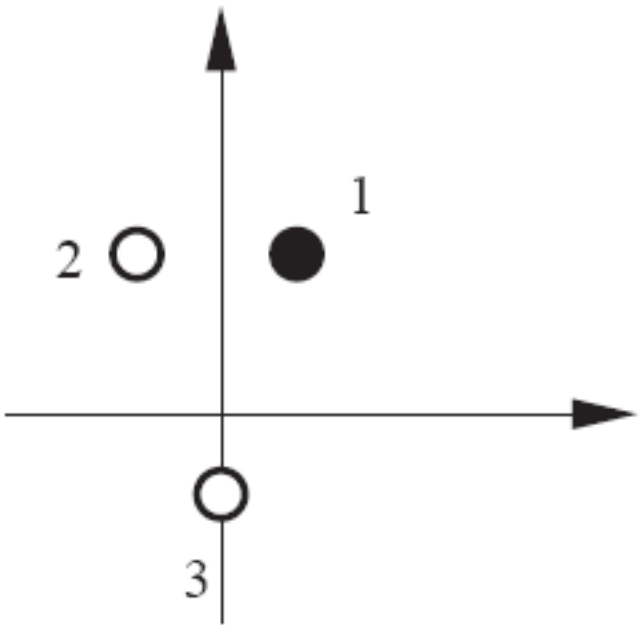
unsupervised learning

The learning rule is an example of supervised training, in which the learning rule is provided with a set of examples of proper network behavior:

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \ldots, \{\mathbf{p}_Q, \mathbf{t}_Q\}$$

Constructing Learning Rules

$$\left\{\mathbf{p}_1 = \begin{bmatrix}1\\2\end{bmatrix}, t_1 = 1\right\}\left\{\mathbf{p}_2 = \begin{bmatrix}-1\\2\end{bmatrix}, t_2 = 0\right\}\left\{\mathbf{p}_3 = \begin{bmatrix}0\\-1\end{bmatrix}, t_3 = 0\right\}.$$

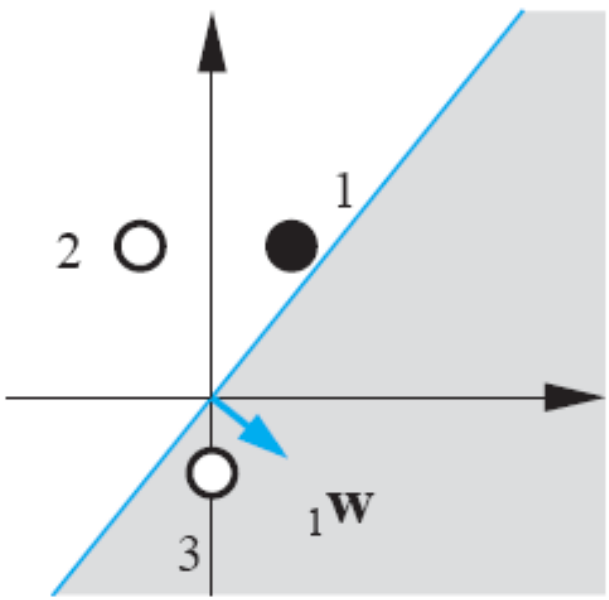


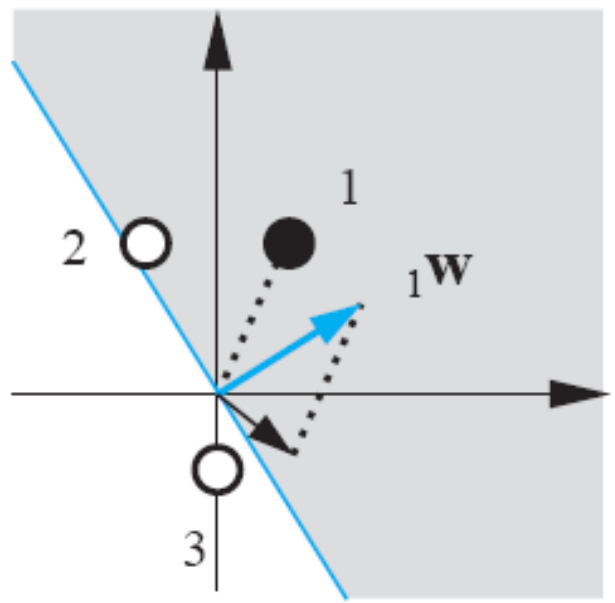
Let ${}_1\mathbf{w}^T = \begin{bmatrix}1.0 & -0.8\end{bmatrix}.$

$$a = hardlim({}_1\mathbf{w}^T\mathbf{p}_1) = hardlim\left(\begin{bmatrix}1.0 & -0.8\end{bmatrix}\begin{bmatrix}1\\2\end{bmatrix}\right)$$

$$a = hardlim(-0.6) = 0\ .$$

If $t = 1$ and $a = 0$, then ${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} + \mathbf{p}$.





Unified Learning Rule

$$e = t - a .$$

$$\text{If } e = 1, \text{ then } {}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} + \mathbf{p} .$$

$$\text{If } e = -1, \text{ then } {}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} - \mathbf{p} .$$

$$\text{If } e = 0, \text{ then } {}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} .$$

$${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} + e\mathbf{p} = {}_1\mathbf{w}^{old} + (t - a)\mathbf{p} .$$

$$b^{new} = b^{old} + e .$$

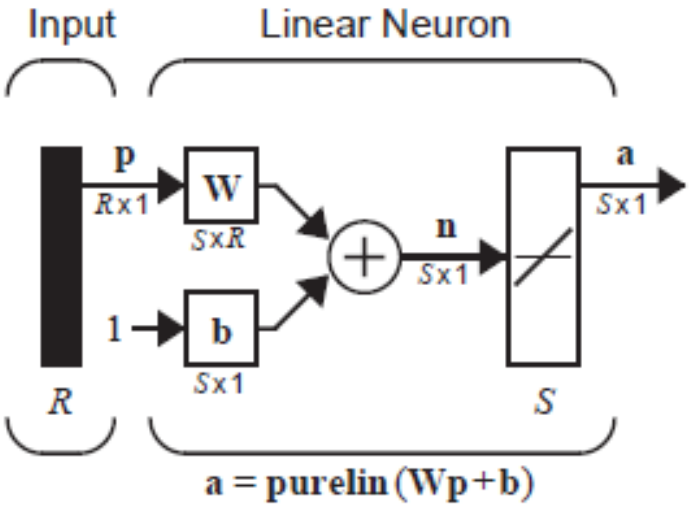
Neural Network

A neural network is a massively parallel distributed processor made up of simple processing unit, which has a natural propensity for storing experimental knowledge and making it available for use. It resemble the brain in two respects

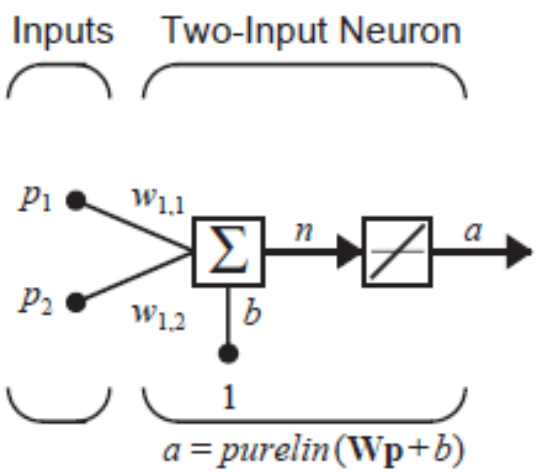
(1)Knowledge is acquired by the network from its environment through a learn process.

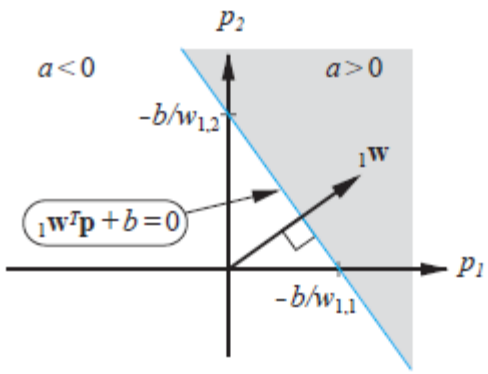
(2)Inter connection strengths, known as synaptic weights are used to store the acquired knowledge

ADALINE



$$\mathbf{a} = \mathbf{purelin}(\mathbf{W}\mathbf{p} + \mathbf{b}) = \mathbf{W}\mathbf{p} + \mathbf{b}\,.$$


Inputs
Two-Input Neuron
p_1
p_2
$w_{1,1}$
$w_{1,2}$
Σ
n
a
b
1
$a = purelin(\mathbf{W}\mathbf{p} + b)$


p_2
$a < 0$
$a > 0$
$-b/w_{1,2}$
${}_1\mathbf{w}$
${}_1\mathbf{w}^T\mathbf{p} + b = 0$
p_1
$-b/w_{1,1}$

Wiener-Hopf equation

- Considering weight and bias in one vector

$$\mathbf{x} = \begin{bmatrix} {}_1\mathbf{w} \\ b \end{bmatrix}.$$

Similarly the bias input "1" as a component of the input vector

$$\mathbf{z} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}.$$

Output of Network

$$a = {}_1\mathbf{w}^T\mathbf{p} + b,$$

$$a = \mathbf{x}^T\mathbf{z}.$$

Network mean square error:

$$F(\mathbf{x}) = E[e^2] = E[(t-a)^2] = E[(t-\mathbf{x}^T\mathbf{z})^2],$$

$$F(\mathbf{x}) = E[t^2 - 2t\mathbf{x}^T\mathbf{z} + \mathbf{x}^T\mathbf{z}\mathbf{z}^T\mathbf{x}]$$

$$= E[t^2] - 2\mathbf{x}^T E[t\mathbf{z}] + \mathbf{x}^T E[\mathbf{z}\mathbf{z}^T]\mathbf{x}.$$

The expression be written in the following convenient form:

$$F(\mathbf{x}) = c - 2\mathbf{x}^T\mathbf{h} + \mathbf{x}^T\mathbf{R}\mathbf{x},$$

Where $$c = E[t^2],\ \mathbf{h} = E[t\mathbf{z}] \text{ and } \mathbf{R} = E[\mathbf{z}\mathbf{z}^T].$$

Comparing with quadratic equation $$F(\mathbf{x}) = c + \mathbf{d}^T\mathbf{x} + \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x}.$$

Where $$\mathbf{d} = -2\mathbf{h} \text{ and } \mathbf{A} = 2\mathbf{R}.$$

Taking gradient $$\nabla F(\mathbf{x}) = \nabla\left(c + \mathbf{d}^T\mathbf{x} + \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x}\right) = \mathbf{d} + \mathbf{A}\mathbf{x} = -2\mathbf{h} + 2\mathbf{R}\mathbf{x}.$$

$$-2\mathbf{h} + 2\mathbf{R}\mathbf{x} = 0.$$

$$\mathbf{x}^* = \mathbf{R}^{-1}\mathbf{h}.$$

Least mean square algorithm

Mean square error $$\hat{F}(\mathbf{x}) = (t(k) - a(k))^2 = e^2(k),$$

Then, at each iteration we have a gradient estimate of the form: $$\hat{\nabla} F(\mathbf{x}) = \nabla e^2(k).$$

Derivative with respect to the Weight $$[\nabla e^2(k)]_j = \frac{\partial e^2(k)}{\partial w_{1,j}} = 2e(k)\frac{\partial e(k)}{\partial w_{1,j}} \text{ for } j = 1, 2, \ldots, R,$$

Derivative with respect to the bias $$[\nabla e^2(k)]_{R+1} = \frac{\partial e^2(k)}{\partial b} = 2e(k)\frac{\partial e(k)}{\partial b}.$$

$$= \frac{\partial}{\partial w_{1,j}}\left[t(k) - \left(\sum_{i=1}^{R} w_{1,i}p_i(k) + b\right)\right]$$

LMS Con..

- We obtain after derivative weight and bias

$$\frac{\partial e(k)}{\partial w_{1,j}} = -p_j(k)\,. \qquad \frac{\partial e(k)}{\partial b} = -1\,.$$

Note that $p_j(k)$ and 1 are the elements of the input vector z , so the gradient of the squared error at iteration can be written

$$\hat{\nabla} F(\mathbf{x}) = \nabla e^2(k) = -2e(k)\mathbf{z}(k)\,.$$

Using steepest descent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla F(\mathbf{x})\big|_{\mathbf{x}=\mathbf{x}_k}\,.$$

If we substitute $\hat{\nabla} F(\mathbf{x})$,

$$\mathbf{x}_{k+1} = \mathbf{x}_k + 2\alpha e(k)\mathbf{z}(k)\,,$$

$$_1\mathbf{w}(k+1) = {}_1\mathbf{w}(k) + 2\alpha e(k)\mathbf{p}(k)\,,$$

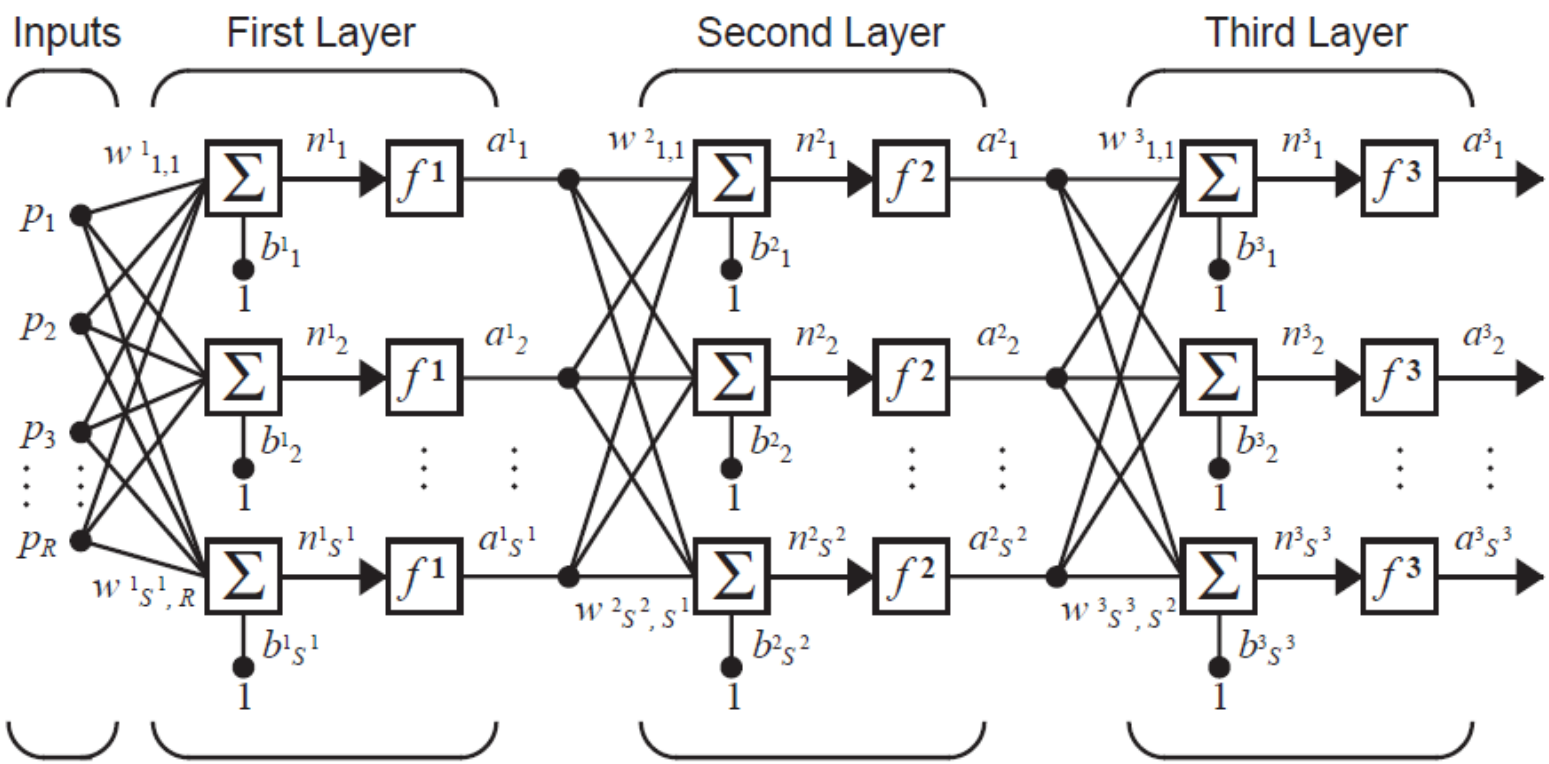
Splitting x into weight and bias

$$b(k+1) = b(k) + 2\alpha e(k)\,.$$

Backpropagation

- The perceptron learning rule of Frank Rosenblatt and the LMS algorithm of Bernard Widrow and Marcian Hoff were designed to train single-layer perceptron-like networks.
- Single-layer networks suffer from the disadvantage that they are only able to solve linearly separable classification problems.
- The difference between the LMS algorithm and backpropagation is only in the way in which the derivatives are calculated.
- For a single-layer linear network the error is an explicit linear function of the network weights, and its derivatives with respect to the weights can be easily computed
- In multilayer networks with nonlinear transfer functions, the relationship between the network weights and the error is more complex.
- In order to calculate the derivatives, we need to use the chain rule of calculus. In fact, here in large part a demonstration of how to use the chain rule.

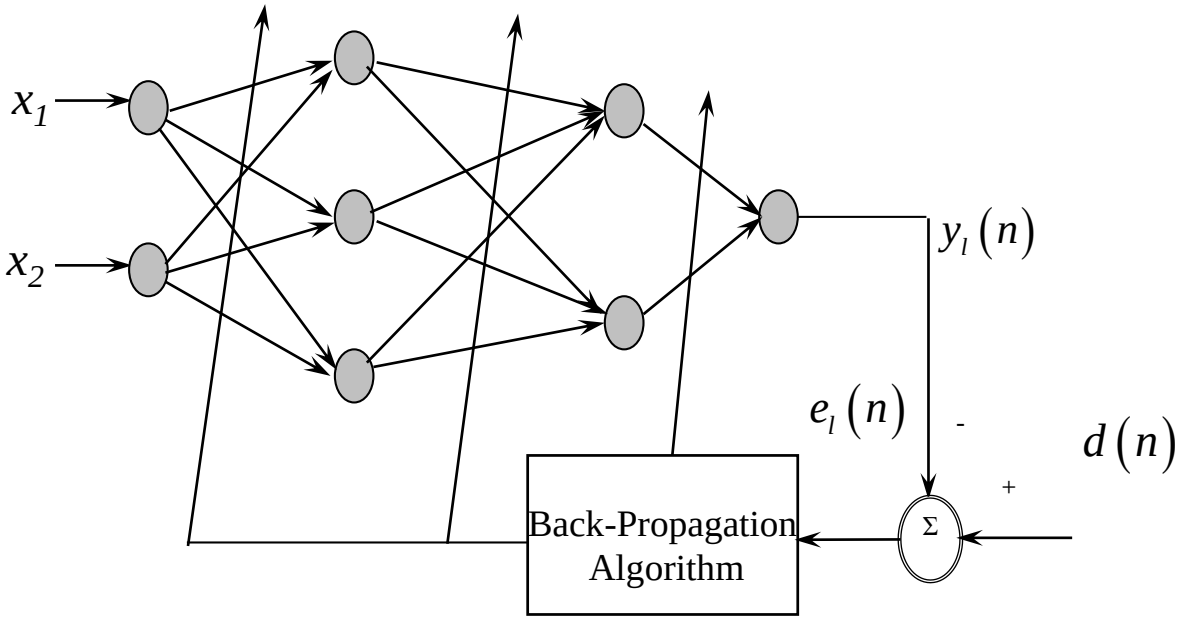
Three layer network



$$\mathbf{a}^1 = \mathbf{f}^1(\mathbf{W}^1\mathbf{p}+\mathbf{b}^1) \qquad \mathbf{a}^2 = \mathbf{f}^2(\mathbf{W}^2\mathbf{a}^1+\mathbf{b}^2) \qquad \mathbf{a}^3 = \mathbf{f}^3(\mathbf{W}^3\mathbf{a}^2+\mathbf{b}^3)$$

$$\mathbf{a}^3 = \mathbf{f}^3(\mathbf{W}^3\mathbf{f}^2(\mathbf{W}^2\mathbf{f}^1(\mathbf{W}^1\mathbf{p}+\mathbf{b}^1)+\mathbf{b}^2)+\mathbf{b}^3)$$

Back Propagation Training



Neural network using BP algorithm

The error signal is given by $e_l(n) = d(n) - y_l(n)$

$$\mathbf{a}^{m+1} = \mathbf{f}^{m+1}(\mathbf{W}^{m+1}\mathbf{a}^{m} + \mathbf{b}^{m+1}) \text{ for } m = 0, 1, \ldots, M-1,$$

M is the layer of the network

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \ldots, \{\mathbf{p}_Q, \mathbf{t}_Q\},$$

Mean square error: $F(\mathbf{x}) = E[e^2] = E[(t-a)^2]$.

If the network has multiple outputs this generalizes to

$$F(\mathbf{x}) = E[\mathbf{e}^T\mathbf{e}] = E[(\mathbf{t}-\mathbf{a})^T(\mathbf{t}-\mathbf{a})].$$

$$\hat{F}(\mathbf{x}) = (\mathbf{t}(k)-\mathbf{a}(k))^T(\mathbf{t}(k)-\mathbf{a}(k)) = \mathbf{e}^T(k)\mathbf{e}(k),$$

Where the expectation of the squared error has been replaced by the squared error at iteration .

- The steepest descent algorithm for the approximate mean square error (stochastic gradient descent) is

$$w_{i,j}^{m}(k+1) = w_{i,j}^{m}(k) - \alpha \frac{\partial \hat{F}}{\partial w_{i,j}^{m}},$$

$$b_{i}^{m}(k+1) = b_{i}^{m}(k) - \alpha \frac{\partial \hat{F}}{\partial b_{i}^{m}},$$

where alpha is the learning rate.

- So far, this development is identical to that for the LMS algorithm. Now we come to the difficult part – the computation of the partial derivatives.

Chain rule

To review the chain rule, suppose that we have a function *f* that is an explicit function only of the variable *n*. We want to take the derivative of with respect to a third variable *w* . The chain rule is then:

$$\frac{df(n(w))}{dw} = \frac{df(n)}{dn} \times \frac{dn(w)}{dw}.$$

For example, if $f(n) = e^{n}$ and $n = 2w$, so that $f(n(w)) = e^{2w}$,

$$\frac{df(n(w))}{dw} = \frac{df(n)}{dn} \times \frac{dn(w)}{dw} = (e^{n})(2).$$

$$\frac{\partial \hat{F}}{\partial w_{i,j}^{m}} = \frac{\partial \hat{F}}{\partial n_{i}^{m}} \times \frac{\partial n_{i}^{m}}{\partial w_{i,j}^{m}},$$

$$\frac{\partial \hat{F}}{\partial b_{i}^{m}} = \frac{\partial \hat{F}}{\partial n_{i}^{m}} \times \frac{\partial n_{i}^{m}}{\partial b_{i}^{m}}.$$

- The second term in each of these equations can be easily computed, since
- the net input to layer is an explicit function of the weights and bias in that layer:

$$n_i^m = \sum_{j=1}^{S^{m-1}} w_{i,j}^m a_j^{m-1} + b_i^m .$$

Therefore

$$\frac{\partial n_i^m}{\partial w_{i,j}^m} = a_j^{m-1} , \quad \frac{\partial n_i^m}{\partial b_i^m} = 1 .$$

If we now define

$$s_i^m \equiv \frac{\partial \hat{F}}{\partial n_i^m} ,$$

$$\frac{\partial \hat{F}}{\partial w_{i,j}^m} = s_i^m a_j^{m-1} ,$$

$$\frac{\partial \hat{F}}{\partial b_i^m} = s_i^m .$$

- We can now express the approximate steepest descent algorithm as

$$w_{i,j}^{m}(k+1) = w_{i,j}^{m}(k) - \alpha s_i^m a_j^{m-1},$$

$$b_i^m(k+1) = b_i^m(k) - \alpha s_i^m.$$

In matrix form this becomes:

$$\mathbf{W}^m(k+1) = \mathbf{W}^m(k) - \alpha \mathbf{s}^m (\mathbf{a}^{m-1})^T,$$

$$\mathbf{b}^m(k+1) = \mathbf{b}^m(k) - \alpha \mathbf{s}^m,$$

where

$$\mathbf{s}^m \equiv \frac{\partial \hat{F}}{\partial \mathbf{n}^m} = \begin{bmatrix} \frac{\partial \hat{F}}{\partial n_1^m} \\ \frac{\partial \hat{F}}{\partial n_2^m} \\ \vdots \\ \frac{\partial \hat{F}}{\partial n_{S^m}^m} \end{bmatrix}.$$

To compute the sensitivities S^m

Lets take Jacobian matrix:

$$\frac{\partial \mathbf{n}^{m+1}}{\partial \mathbf{n}^{m}} \equiv \begin{bmatrix} \frac{\partial n_1^{m+1}}{\partial n_1^m} & \frac{\partial n_1^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_1^{m+1}}{\partial n_{S^m}^m} \\ \frac{\partial n_2^{m+1}}{\partial n_1^m} & \frac{\partial n_2^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_2^{m+1}}{\partial n_{S^m}^m} \\ \vdots & \vdots & & \vdots \\ \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_1^m} & \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_{S^m}^m} \end{bmatrix}.$$

Next we want to find an expression for this matrix.
Consider the i, j element of the matrix:

$$\frac{\partial n_i^{m+1}}{\partial n_j^m} = \frac{\partial \left(\sum_{l=1}^{S^m} w_{i,l}^{m+1} a_l^m + b_i^{m+1} \right)}{\partial n_j^m} = w_{i,j}^{m+1} \frac{\partial a_j^m}{\partial n_j^m}$$

where $\dot{f}^m(n_j^m) = \frac{\partial f^m(n_j^m)}{\partial n_j^m}$.

$$= w_{i,j}^{m+1} \frac{\partial f^m(n_j^m)}{\partial n_j^m} = w_{i,j}^{m+1} \dot{f}^m(n_j^m),$$

- Therefore the Jacobian matrix can be written $$\frac{\partial \mathbf{n}^{m+1}}{\partial \mathbf{n}^{m}} = \mathbf{W}^{m+1}\dot{\mathbf{F}}^{m}(\mathbf{n}^{m})$$

$$\dot{\mathbf{F}}^{m}(\mathbf{n}^{m}) = \begin{bmatrix} \dot{f}^{m}(n_1^m) & 0 & \dots & 0 \\ 0 & \dot{f}^{m}(n_2^m) & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \dot{f}^{m}(n_{S^m}^m) \end{bmatrix}.$$

$$\mathbf{s}^{m} = \frac{\partial \hat{F}}{\partial \mathbf{n}^{m}} = \left(\frac{\partial \mathbf{n}^{m+1}}{\partial \mathbf{n}^{m}}\right)^{T} \frac{\partial \hat{F}}{\partial \mathbf{n}^{m+1}} = \dot{\mathbf{F}}^{m}(\mathbf{n}^{m})(\mathbf{W}^{m+1})^{T} \frac{\partial \hat{F}}{\partial \mathbf{n}^{m+1}}$$

$$= \dot{\mathbf{F}}^{m}(\mathbf{n}^{m})(\mathbf{W}^{m+1})^{T}\mathbf{s}^{m+1}.$$

- Now we can see where the backpropagation algorithm derives its name. The sensitivities are propagated backward through the network from last layer to the first layer:

$$\mathbf{s}^{M} \rightarrow \mathbf{s}^{M-1} \rightarrow \dots \rightarrow \mathbf{s}^{2} \rightarrow \mathbf{s}^{1}.$$

To compute s^m

$$s_i^M = \frac{\partial \hat{F}}{\partial n_i^M} = \frac{\partial (\mathbf{t}-\mathbf{a})^T(\mathbf{t}-\mathbf{a})}{\partial n_i^M} = \frac{\overset{s^M}{\partial \sum_{j=1} (t_j - a_j)^2}}{\partial n_i^M} = -2(t_i - a_i)\frac{\partial a_i}{\partial n_i^M}.$$

Now, since

$$\frac{\partial a_i}{\partial n_i^M} = \frac{\partial a_i^M}{\partial n_i^M} = \frac{\partial f^M(n_i^M)}{\partial n_i^M} = \dot{f}^M(n_i^M)$$

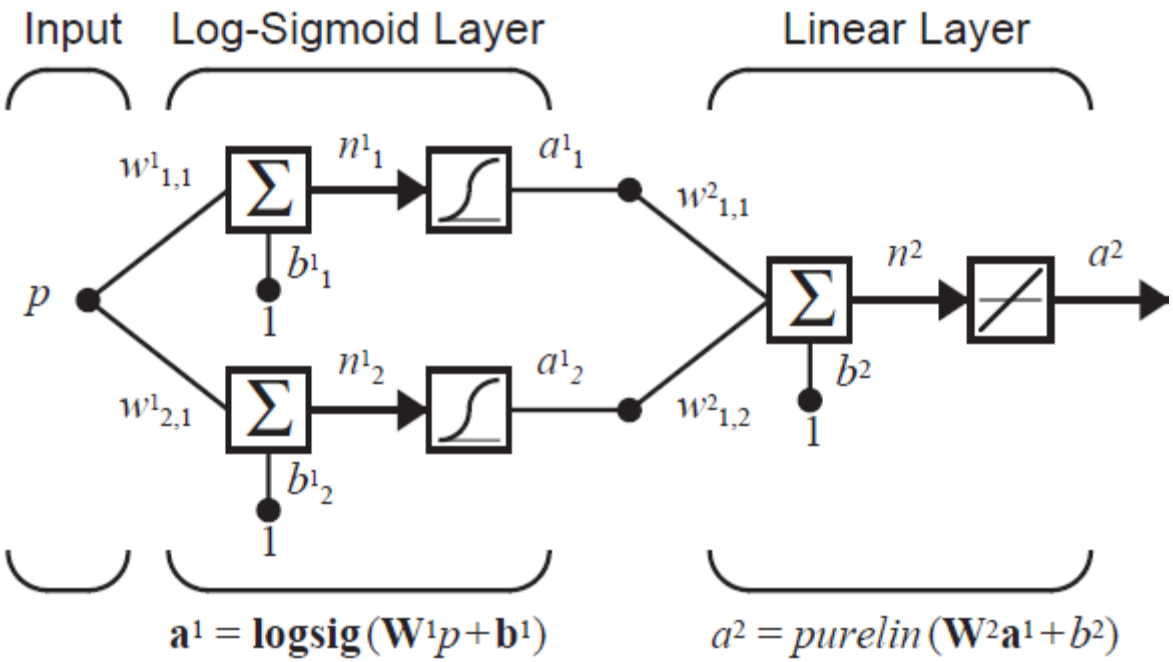
we can write

$$s_i^M = -2(t_i - a_i)\dot{f}^M(n_i^M)$$

This can be expressed in matrix form as

$$\mathbf{s}^M = -2\dot{\mathbf{F}}^M(\mathbf{n}^M)(\mathbf{t}-\mathbf{a})$$

Example on Backpropagation



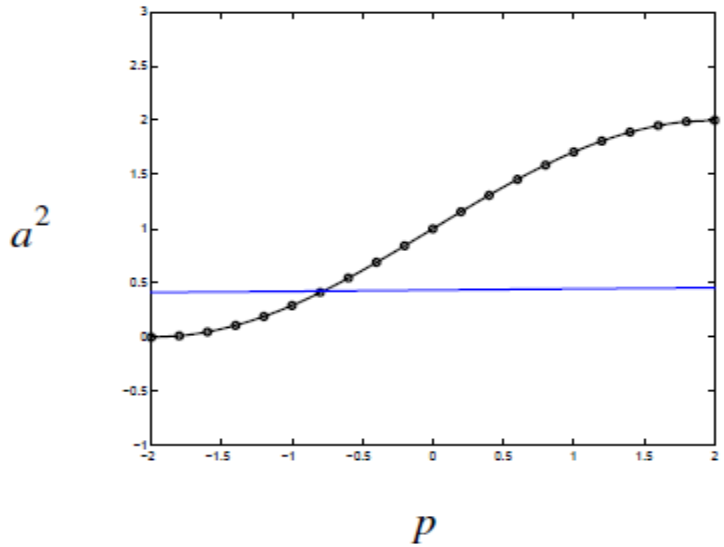
We want to approximate the function

$$g(p) = 1 + \sin\left(\frac{\pi}{4}p\right) \text{ for } -2 \le p \le 2 .$$

Before we begin the backpropagation algorithm we need to choose some initial values for the network weights and biases. Generally these are chosen to be small random values.

$$\mathbf{W}^1(0) = \begin{bmatrix} -0.27 \\ -0.41 \end{bmatrix}, \mathbf{b}^1(0) = \begin{bmatrix} -0.48 \\ -0.13 \end{bmatrix}, \mathbf{W}^2(0) = \begin{bmatrix} 0.09 & -0.17 \end{bmatrix}, \mathbf{b}^2(0) = \begin{bmatrix} 0.48 \end{bmatrix}$$

The response of the network for these initial values is illustrated in Figure



- Next, we need to select a training set .$\{p_1, t_1\}$, $\{p_2, t_2\}$, … , $\{p_Q, t_Q\}$
- In this case, we will sample the function at 21 points in the range [-2,2] at equally spaced intervals of 0.2. The training points are indicated by the circles in previous Fig.
- For our initial input we will choose *p=1*, which is the 16th training point:
- The output of the first layer is then

$$\mathbf{a}^1 = \mathbf{f}^1(\mathbf{W}^1\mathbf{a}^0 + \mathbf{b}^1) = \mathbf{logsig}\left(\begin{bmatrix}-0.27\\-0.41\end{bmatrix}\begin{bmatrix}1\end{bmatrix} + \begin{bmatrix}-0.48\\-0.13\end{bmatrix}\right) = \mathbf{logsig}\left(\begin{bmatrix}-0.75\\-0.54\end{bmatrix}\right)$$

$$= \begin{bmatrix}\dfrac{1}{1+e^{0.75}}\\ \dfrac{1}{1+e^{0.54}}\end{bmatrix} = \begin{bmatrix}0.321\\0.368\end{bmatrix}.$$

- The second layer output is

$$a^2 = f^2(\mathbf{W}^2\mathbf{a}^1 + \mathbf{b}^2) = purelin\left(\begin{bmatrix}0.09 & -0.17\end{bmatrix}\begin{bmatrix}0.321\\0.368\end{bmatrix} + \begin{bmatrix}0.48\end{bmatrix}\right) = \begin{bmatrix}0.446\end{bmatrix}$$

The error would then be

$$e = t - a = \left\{1 + \sin\left(\frac{\pi}{4}p\right)\right\} - a^2 = \left\{1 + \sin\left(\frac{\pi}{4}1\right)\right\} - 0.446 = 1.261$$

The next stage of the algorithm is to backpropagate the sensitivities. Before we begin the backpropagation, recall that we will need the derivatives of the transfer functions $\dot{f}^1(n)$ and $\dot{f}^2(n)$

$$\dot{f}^1(n) = \frac{d}{dn}\left(\frac{1}{1+e^{-n}}\right) = \frac{e^{-n}}{(1+e^{-n})^2} = \left(1 - \frac{1}{1+e^{-n}}\right)\left(\frac{1}{1+e^{-n}}\right) = (1-a^1)(a^1)$$

$$\dot{f}^2(n) = \frac{d}{dn}(n) = 1$$

We can now perform the backpropagation. The starting point is found at the second layer,

$$\mathbf{s}^2 = -2\dot{\mathbf{F}}^2(\mathbf{n}^2)(\mathbf{t}-\mathbf{a}) = -2\left[\dot{f}^2(n^2)\right](1.261) = -2\left[1\right](1.261) = -2.522\,.$$

The first layer sensitivity is then computed by backpropagating the sensitivity from the second layer,

$$\mathbf{s}^1 = \dot{\mathbf{F}}^1(\mathbf{n}^1)(\mathbf{W}^2)^T\mathbf{s}^2 = \begin{bmatrix} (1-a_1^1)(a_1^1) & 0 \\ 0 & (1-a_2^1)(a_2^1) \end{bmatrix} \begin{bmatrix} 0.09 \\ -0.17 \end{bmatrix} \begin{bmatrix} -2.522 \end{bmatrix}$$

$$= \begin{bmatrix} (1-0.321)(0.321) & 0 \\ 0 & (1-0.368)(0.368) \end{bmatrix} \begin{bmatrix} 0.09 \\ -0.17 \end{bmatrix} \begin{bmatrix} -2.522 \end{bmatrix}$$

$$= \begin{bmatrix} 0.218 & 0 \\ 0 & 0.233 \end{bmatrix} \begin{bmatrix} -0.227 \\ 0.429 \end{bmatrix} = \begin{bmatrix} -0.0495 \\ 0.0997 \end{bmatrix}$$

- The final stage of the algorithm is to update the weights. For simplicity, we will use a learning rate *α=0.1*

$$\mathbf{W}^2(1) = \mathbf{W}^2(0) - \alpha \mathbf{s}^2(\mathbf{a}^1)^T = \begin{bmatrix} 0.09 & -0.17 \end{bmatrix} - 0.1 \begin{bmatrix} -2.522 \end{bmatrix} \begin{bmatrix} 0.321 & 0.368 \end{bmatrix}$$

$$= \begin{bmatrix} 0.171 & -0.0772 \end{bmatrix}$$

$$\mathbf{b}^2(1) = \mathbf{b}^2(0) - \alpha \mathbf{s}^2 = \begin{bmatrix} 0.48 \end{bmatrix} - 0.1 \begin{bmatrix} -2.522 \end{bmatrix} = \begin{bmatrix} 0.732 \end{bmatrix},$$

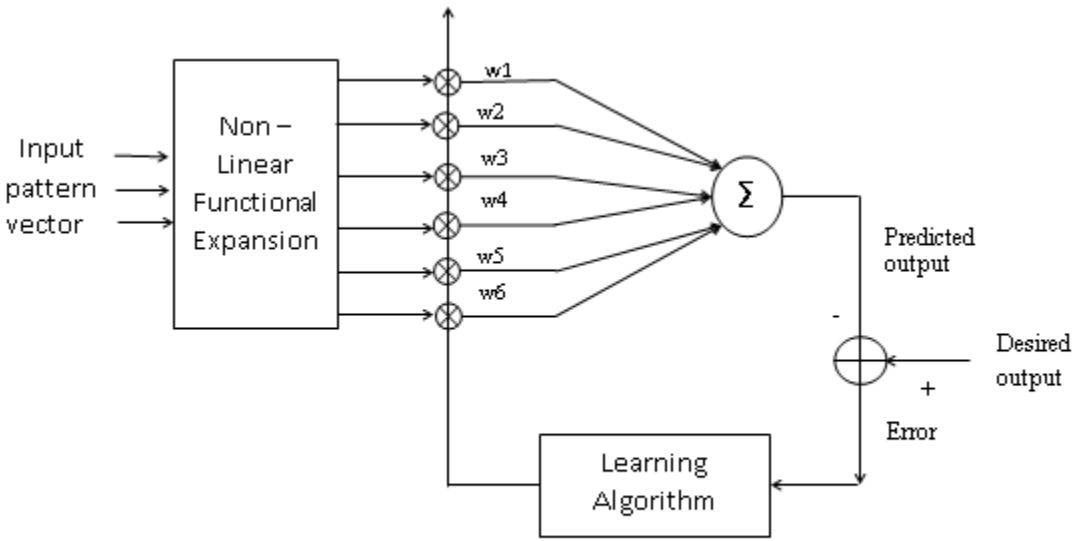
$$\mathbf{W}^1(1) = \mathbf{W}^1(0) - \alpha \mathbf{s}^1(\mathbf{a}^0)^T = \begin{bmatrix} -0.27 \\ -0.41 \end{bmatrix} - 0.1 \begin{bmatrix} -0.0495 \\ 0.0997 \end{bmatrix} \begin{bmatrix} 1 \end{bmatrix} = \begin{bmatrix} -0.265 \\ -0.420 \end{bmatrix}$$

$$\mathbf{b}^1(1) = \mathbf{b}^1(0) - \alpha \mathbf{s}^1 = \begin{bmatrix} -0.48 \\ -0.13 \end{bmatrix} - 0.1 \begin{bmatrix} -0.0495 \\ 0.0997 \end{bmatrix} = \begin{bmatrix} -0.475 \\ -0.140 \end{bmatrix}.$$

This completes the first iteration of the backpropagation algorithm. We next proceed to randomly choose another input from the training set and perform another iteration of the algorithm. We continue to iterate until the difference between the network response and the target function reaches some acceptable level.

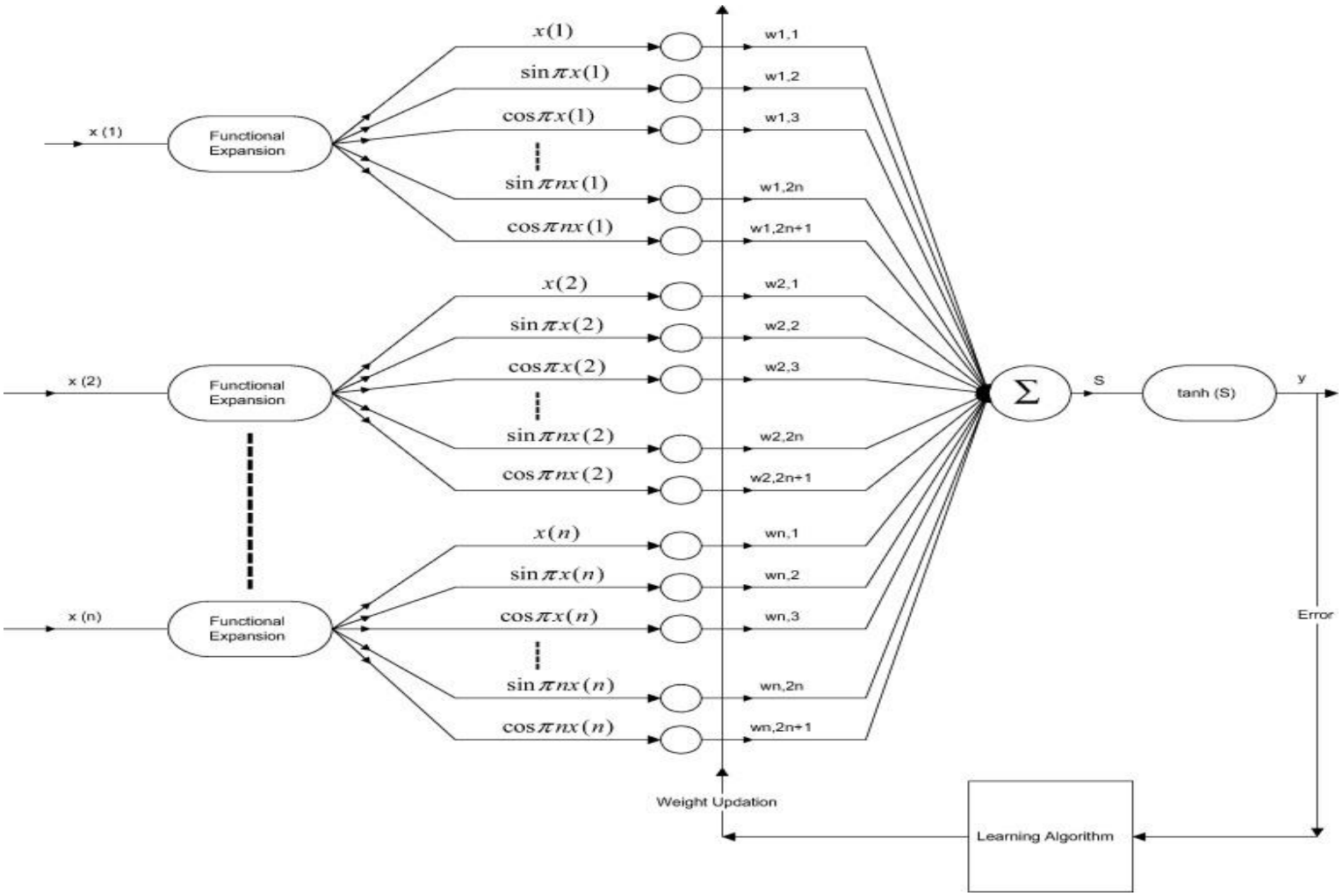
FLANN (Functional Link ANN)

- MLP has high computation complexity as it has many layers
- The FLANN is basically a single layer structure in which non-linearity is introduced by enhancing the input pattern with nonlinear function expansion.



Block Diagram of FLANN model

Trigonometric FLANN



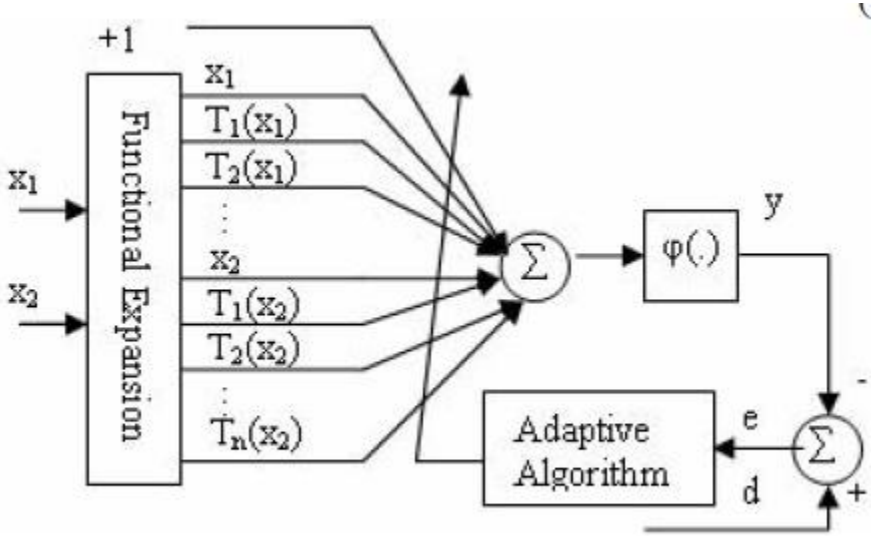
Chebyshev FLANN

These higher Chebyshev polynomials for -1<x<1 may be generated using the recursive formula given by

$$T_{n+1} = 2xT_n(x) - T_{n-1}(x)$$

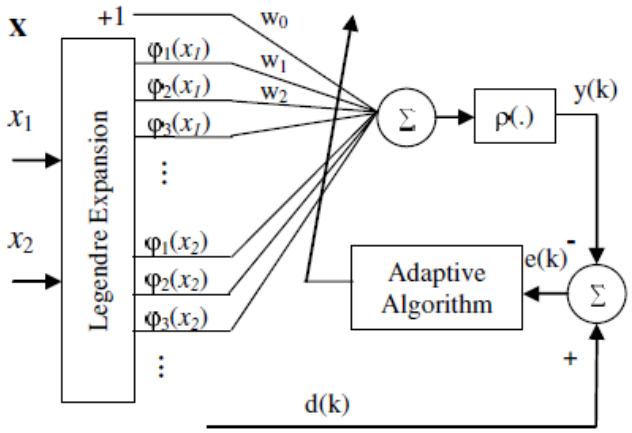
The first few Chebyshev polynomials are given by

$$\left.\begin{aligned} T_0(x) &= 1 \\ T_1(x) &= x \\ T_2(x) &= 2x^2-1 \\ T_3(x) &= 4x^3-3x \\ T_4(x) &= 8x^4-8x^2+1 \\ T_5(x) &= 16x^5-20x^3+5x \end{aligned}\right\}$$



Legendre-FLANN

$$P_0(x) = 1$$
$$P_1(x) = x$$
$$P_2(x) = \frac{1}{2}(3x^2 - 1)$$
$$P_3(x) = \frac{1}{2}(5x^3 - 3x)$$
$$P_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3).$$

The higher order Legendre polynomials can be derived from the following recursion formula:

$$P_{n+1}(x) = \frac{1}{n+1}[(2n+1)xP_n(x) - nP_{n-1}(x)]$$